Akka Classic — Fast Exam Reference & Code Playbook (NSDS)

1. Exam Decision Matrix — "Which Pattern Do I Need?"

Exam Requirement	Akka Mechanism to Use	Jump to
Filter messages by type vs. by field value	`.match(Class, handler)` vs. `.match(Class, msg -> cond, handler)`	Pattern 1
Pass `ActorRef` or config parameters to an actor constructor	`Props.create(MyActor.class, () -> new MyActor(arg1, arg2))` (or via a `ConfigMsg`)	Pattern 1
Actor changes how it reacts to messages based on its mode/state	`getContext().become(nextBehavior())` (Finite State Machine)	Pattern 2
Postpone messages when a precondition fails (e.g., `counter == 0` or uninitialized)	Extend `AbstractActorWithStash` → `stash()` and `unstashAll()`	Pattern 3
Server enters **Sleep/Pause mode**, buffers requests, and drains them on **Wakeup**	`AbstractActorWithStash` + `become(onSleepMode())` + `stash()` → `unstashAll()` + `become(createReceive())` (Lab 2 `ex4`)	Pattern 3B
Wait for a reply **without blocking** the actor thread	`AbstractActorWithStash` + `become(waiting())` + `.matchAny(m -> stash())`	Pattern 4A
Wait for a reply **synchronously** (blocking)	`Patterns.ask(target, msg, timeoutMs)` + `Await.result(future, duration)`	Pattern 4B
Simulate **non-blocking processing delay** while buffering concurrent arrivals	`scheduler().scheduleOnce(delay, self(), new ReadyMsg(sender(), res), dispatcher(), self())` + `become(onBusyMode())` + `stash()` (Lab 2 `ex6`)	Pattern 4C
Forward message through a router/proxy keeping original sender	`target.tell(msg, sender())` (or `target.forward(msg, getContext())`)	Section 2
Handle actor crashes (`resume` to keep state vs. `restart` to reset state)	Parent overrides `supervisorStrategy()` with `OneForOneStrategy` / `AllForOneStrategy` (Lab 2 `ex5`)	Pattern 5
Protect critical state while running a fault-prone computation	Keep critical state in **Parent Supervisor**, delegate risky task to **Child Worker**	Pattern 5B
Dynamic routing/dispatching (Round-Robin vs. Sticky Load Balancing) + Supervision	Dispatcher acts as **Supervisor** of worker actors (`getContext().actorOf`) and switches policy via `become()`	Pattern 7
Connect Client and Server across separate processes/machines	`ConfigFactory.parseFile` (`remote.conf`) + `getContext().actorSelection("akka.tcp:// ... ")`	Pattern 6

2. Essential Imports & One-Liner API Cheat Sheet

```
// --- IMPORTS ---
import akka.actor.*;
import akka.japi.pf.DeciderBuilder;
import akka.pattern.Patterns;
```

```
import com.typesafe.config.Config;
import com.typesafe.config.ConfigFactory;
import scala.concurrent.Await;
import scala.concurrent.Future;
import java.io.File;
import java.io.Serializable;
import java.time.Duration;
import java.util.*;
import java.util.concurrent.*;
import static akka.pattern.Patterns.ask;
import static java.util.concurrent.TimeUnit.SECONDS;

// --- SYSTEM & ACTOR CREATION ---
ActorSystem sys = ActorSystem.create("System");
ActorRef actor = sys.actorOf(MyActor.props(arg), "myActor");         // Top-level actor
ActorRef child = getContext().actorOf(ChildActor.props(), "child");  // Supervised child actor (inside
parent!)

// --- SENDING, REPLYING & FORWARDING ---
actor.tell(new Msg(), ActorRef.noSender()); // From main() (no sender)
actor.tell(new Msg(), self());              // From inside an actor (self() = getSelf())
sender().tell(new ReplyMsg(), self());      // Reply to whoever sent current message (sender() = getSender())
target.tell(msg, sender());                 // Forward preserving original sender()

// --- NON-BLOCKING TIMER / SCHEDULER (Lab 2 Ex 6) ---
// Schedules 'msg' to be sent to 'recipient' after 'delay' WITHOUT blocking the actor's thread
getContext().getSystem().scheduler().scheduleOnce(
        Duration.ofMillis(500),             // Delay (java.time.Duration)
        self(),                             // Recipient ActorRef (usually self())
        new ReadyMessage(getSender(), res), // Message to deliver when timer expires (save original sender!)
        getContext().dispatcher(),          // ExecutionContext / Dispatcher
        self()                              // Sender ActorRef
);

// --- STOPPING ---
getContext().stop(child); // Stop a child actor
getContext().stop(self()); // Stop self
sys.terminate();          // Shut down entire ActorSystem

// --- SYNCHRONOUS ASK (TWO EQUIVALENT FORMS) ---
// Form 1: Patterns.ask + Await.result
Future<Object> f = Patterns.ask(server, new RequestMsg(), 5000);
ReplyMsg rep = (ReplyMsg) Await.result(f, scala.concurrent.duration.Duration.create(5, SECONDS));

// Form 2: ask + future.result (often used in main() to get a supervised child ActorRef from a Supervisor -
Lab 2 Ex 5)
scala.concurrent.duration.Duration timeout = scala.concurrent.duration.Duration.create(5, SECONDS);
Future<Object> fChild = ask(supervisor, Props.create(ServerActor.class), 5000);
ActorRef server = (ActorRef) fChild.result(timeout, null);
```

3. Core Theory Rules (Rapid Review for Open/Multiple-Choice Questions)

3.1 Concurrency, Atomicity & Mailbox Semantics

• **3 Actor Actions on Message Receive**: (1) local decisions (mutate private state or `become()`), (2) send *0, 1,* or more messages, (3) spawn child actors.

• **Why Locks Are Never Needed**: Actor state is private. Messages are processed **one at a time atomically** (*as if inside a* `synchronized` *method of a class where all methods are* `synchronized`). Concurrent arrivals wait in the mailbox.

• **Why** `Thread.sleep()` **Inside an Actor Is Wrong (Lab 2 Ex 6)**: Calling `Thread.sleep()` blocks the underlying dispatcher thread and prevents the actor from inspecting new messages (e.g., to `stash()` them or react to control signals). Always use `getContext().getSystem().scheduler().scheduleOnce( ... )` + `become(busy)` + `stash()`.

• **Pattern Matching Order**: `createReceive()` evaluates `.match( ... )` clauses **top-to-bottom** and runs the **first match**. If **no clause matches**, the message is **silently discarded**.

• **Ordering Guarantees**: Default mailbox is **FIFO**.

  • **Single sender → receiver**: **FIFO order** guaranteed (over local memory or TCP).

  - **Multiple senders → receiver**: arrival interleaving is **non-deterministic**.
- **Immutability**: Messages **must be immutable** (`private final` fields, `Collections.unmodifiableList( ... )` for collections). The framework **cannot check this statically** at compile time, but mutable messages cause data races.

3.2 Lifecycle, Addressing & Communication

- **`ActorPath` vs. `ActorRef` vs. `ActorSelection`**:
  - **`ActorPath`**: logical hierarchical name. Freed and reusable after the actor stops.
  - **`ActorRef`**: points to a specific **incarnation** (`ActorPath` + random `UID`). **Remains valid across restarts** and can be sent inside messages (e.g., `ConfigurationMessage` in Lab 2 `ex4` / `ex6`).
  - **`ActorSelection`**: path query (supports wildcards); resolves the underlying `ActorRef` via `Identify` → `ActorIdentity` .
  - **Many-to-Many**: 1 address can represent multiple actors (load-balancing/routing); 1 actor can have multiple addresses.
- **Lifecycle Hooks**:
  - `preStart()` : called right after creation, before first message (ideal place for a Supervisor/Dispatcher to spawn child workers via `getContext().actorOf( ... )`).
  - `preRestart(reason, msg)` : called on the **old** crashing instance before `restart()` . (**Not** called on `resume()` !).
  - `postRestart(reason)` : called on the **new** instance after `restart()` . (**Not** called on `resume()` !).
  - `postStop()` : called when permanently stopped (`stop()` , `PoisonPill` , `sys.terminate()`); sends `Terminated` to watchers.
- **Best-Effort Communication ("Less is more")**:
  - Direct point-to-point (no mediating channel). **At-most-once** (no duplicates, messages can be lost) and no global order.
  - *Why?* IP is best-effort, TCP cannot mask network partitions, and stronger guarantees are expensive/not always needed (can be built at application level).

3.3 Stash, Fault Tolerance & Clustering

- **Stash (`AbstractActorWithStash`)**:
  - `stash()` buffers the current message (preserving its original `sender()` reference!). `unstashAll()` extracts all stashed messages **in FIFO order** and **prepends them to the front of the mailbox** (processed before newer mailbox messages).
  - *Always call* `unstashAll()` *right before* `getContext().become(normalBehavior())` when leaving a sleeping/waiting/busy state.
- **Supervision & Restart Rules (Lab 2 Ex 5)**:
  - Every actor is supervised by its **parent** (the actor that called `getContext().actorOf()`). Top-level actors (`sys.actorOf()`) are supervised by the system's `/user` guardian, **not** by your custom supervisor!
  - **Hierarchy of Responsibility**: keep critical data in the **supervisor**; delegate dangerous subtasks to **children**. A supervisor strategy depends only on exception type and error count (not which child failed), so supervise homogeneous children (build deeper trees for different roles).
  - **`resume()` vs. `restart()` (Lab 2 Ex 5)**:
    - **`SupervisorStrategy.resume()`**: the crashed actor instance is kept alive and **its internal state (e.g., contact list `HashMap` or running average) is preserved**, while the faulty message that threw the exception is discarded.
    - **`SupervisorStrategy.restart()`**: the crashed actor instance is replaced with a **new instance** whose **internal state is reset to initial constructor values**, while its **`ActorRef` (address) does NOT change** and its **mailbox is preserved** (lives outside actor memory).
- **Strategies & Directives**:
  - `OneForOneStrategy` (applies directive only to the crashed child) vs. `AllForOneStrategy` (applies directive to the crashed child + all its siblings).
  - Directives: `resume()` , `restart()` , `stop()` , `escalate()` .
  - If faults exceed `maxNrOfRetries` within `withinTimeRange` , the child is **stopped**.
- **Clustering**:
  - **Node**: `hostname:port:uid` (process/ `ActorSystem`); membership is decoupled from hosted actors.
  - **Protocol**: Decentralized P2P overlay using **gossip** with **vector clocks** (partial event ordering) + automatic failure detection.

• **Gossip Convergence**: reached when a node proves its observed state matches all other nodes. **Blocked while any node is** `"unreachable"` (must become reachable or be removed). Managed by the **Leader** role (not a SPOF).

• **Seed Nodes**: joining node contacts **all seed nodes** (at least one active) and sends `join` to the **first seed node that replies**.

• **Cluster Tools**: *Cluster Singleton* (1 instance in cluster), *Cluster Sharding* (distribute by logical ID), *Distributed Data* (key-value CRDT store), *Cluster Metrics* (health/load monitoring).

---

4. Complete Commented Code Patterns

Pattern 1 — Immutable Messages, Basic Actor & Predicates

(Ref: `TemplateMessage.java`, `TemplateActor.java`, *Lab 1* `ex1` *)*

```
// 1. IMMUTABLE MESSAGE (final fields, defensive copy for collections, Serializable)
public class OpMsg implements Serializable {
    private static final long serialVersionUID = 1L;
    public enum Type { INC, DEC }

    private final Type type;
    private final int amount;
    private final List<String> tags;

    public OpMsg(Type type, int amount, List<String> tags) {
        this.type = type;
        this.amount = amount;
        // Defensive copy so sender cannot mutate the list after sending
        this.tags = Collections.unmodifiableList(new ArrayList<>(tags));
    }

    public Type getType() { return type; }
    public int getAmount() { return amount; }
    public List<String> getTags() { return tags; }
}

// 2. STATEFUL ACTOR WITH PREDICATE & CLASS MATCHING
public class CounterActor extends AbstractActor {
    private int counter; // Private state: thread-safe without locks!

    public CounterActor(int initialValue) {
        this.counter = initialValue;
    }

    @Override
    public Receive createReceive() {
        // Clauses evaluated top-to-bottom: put specific predicates BEFORE general class matches!
        return receiveBuilder()
                // Match by class + predicate selector
                .match(OpMsg.class, m -> m.getType() == OpMsg.Type.INC && m.getAmount() > 0, this::onInc)
                .match(OpMsg.class, m -> m.getType() == OpMsg.Type.DEC && m.getAmount() > 0, this::onDec)
                // Match by class only
                .match(String.class, cmd -> {
                    // Reply to sender with current state
                    sender().tell(counter, self());
                })
                .build();
    }

    private void onInc(OpMsg msg) { counter += msg.getAmount(); }
    private void onDec(OpMsg msg) { counter -= msg.getAmount(); }

    // Factory method with constructor argument
    public static Props props(int initialValue) {
        return Props.create(CounterActor.class, () -> new CounterActor(initialValue));
    }
}
```

Pattern 2 — Dynamic Behavior / State Machine (`become`)

(Ref: `StateMachineActorTemplate.java`)

```
public class AlarmActor extends AbstractActor {
    public static class ActivateMsg implements Serializable {}
    public static class DeactivateMsg implements Serializable {}
    public static class TriggerMsg implements Serializable {}

    @Override
    public Receive createReceive() {
        return disabled(); // Initial behavior state
    }

    // State 1: DISABLED (TriggerMsg is not matched → automatically discarded!)
    private Receive disabled() {
        return receiveBuilder()
                .match(ActivateMsg.class, msg → {
                    System.out.println("Switched to ENABLED");
                    getContext().become(enabled()); // Transition to enabled()
                })
                .build();
    }

    // State 2: ENABLED
    private Receive enabled() {
        return receiveBuilder()
                .match(TriggerMsg.class, msg → System.out.println("ALARM!"))
                .match(DeactivateMsg.class, msg → {
                    System.out.println("Switched to DISABLED");
                    getContext().become(disabled()); // Transition to disabled()
                })
                .build();
    }

    public static Props props() { return Props.create(AlarmActor.class); }
}
```

Pattern 3 — Conditional Processing with `stash()` & `unstashAll()`

(Ref: `StashActorTemplate.java`, Lab 1 `ex2`, Lab 2 `ex4`)

```
// MUST extend AbstractActorWithStash (NOT AbstractActor)
public class NonNegativeCounterActor extends AbstractActorWithStash {
    public static class IncMsg implements Serializable {}
    public static class DecMsg implements Serializable {}

    private int counter = 0;

    @Override
    public Receive createReceive() {
        return receiveBuilder()
                .match(IncMsg.class, this::onInc)
                .match(DecMsg.class, this::onDec)
                .build();
    }

    private void onInc(IncMsg msg) {
        ++counter;
        System.out.println("Counter = " + counter);
        // State changed → prepend all stashed DecMsg back to the front of mailbox in FIFO order!
        unstashAll();
    }

    private void onDec(DecMsg msg) {
        if (counter > 0) {
            --counter;
            System.out.println("Counter = " + counter);
        } else {
```

```
            // Precondition failed (counter == 0) → postpone message in stash buffer
            stash();
        }
    }

    public static Props props() { return Props.create(NonNegativeCounterActor.class); }
}
```

Pattern 3B — Sleep/Wakeup Buffering Server (Lab 2 `ex4`)

(Ref: Lab 2 `ex4/ServerActor.java` & `ex4/ClientActor.java`)

```
// Server echoes TextMessage when awake; on SleepMessage it buffers TextMessages via stash();
// on WakeupMessage it flushes accumulated messages in FIFO order via unstashAll() and resumes echoing.
public class SleepWakeServerActor extends AbstractActorWithStash {

    @Override
    public Receive createReceive() {
        // Awake mode: echo TextMessages immediately and listen for SleepMessage
        return receiveBuilder()
                .match(TextMessage.class, this::onTextMessage)
                .match(SleepMessage.class, this::onSleepMessage)
                .build();
    }

    private void onTextMessage(TextMessage msg) {
        // Echo back to the client that sent it (sender() is preserved even after stash/unstashAll!)
        sender().tell(msg, ActorRef.noSender());
    }

    private void onSleepMessage(SleepMessage msg) {
        getContext().become(onSleepMode()); // Enter sleep mode
    }

    private Receive onSleepMode() {
        // Sleep mode: stash incoming TextMessages and wait for WakeupMessage
        return receiveBuilder()
                .match(TextMessage.class, msg → stash())
                .match(WakeupMessage.class, this::onWakeMessage)
                .build();
    }

    private void onWakeMessage(WakeupMessage msg) {
        unstashAll();                       // 1. Prepend accumulated messages back to mailbox in FIFO order
        getContext().become(createReceive()); // 2. Return to awake mode so unstashed messages are echoed
    }

    public static Props props() { return Props.create(SleepWakeServerActor.class); }
}
```

Pattern 4 — Client-Server Request-Reply & Non-Blocking Scheduled Processing

(Ref: `RequestReplyTemplate.java`, Lab 1 `ex3`, Lab 2 `ex6`)

```
// --- SERVER ACTOR (Works identically with both AsyncClient and AskClient) ---
public class ServerActor extends AbstractActor {
    private final Map<String, String> db = new HashMap<>();

    @Override
    public Receive createReceive() {
        return receiveBuilder()
                .match(PutMsg.class, m → db.put(m.getKey(), m.getVal()))
                .match(GetMsg.class, m → {
                    // Always reply to sender() passing self()
                    sender().tell(new ReplyMsg(m.getKey(), db.get(m.getKey())), self());
                })
                .build();
    }
    public static Props props() { return Props.create(ServerActor.class); }
```

```
}

// --- 4A. NON-BLOCKING CLIENT (become + stash + unstashAll) ---
public class AsyncClientActor extends AbstractActorWithStash {
    private final ActorRef server;
    public AsyncClientActor(ActorRef server) { this.server = server; }

    @Override
    public Receive createReceive() { return idle(); }

    private Receive idle() {
        return receiveBuilder()
                .match(PutMsg.class, m -> server.tell(m, self()))
                .match(GetMsg.class, m -> {
                    server.tell(m, self());                  // 1. Send async request
                    getContext().become(waitingForReply());  // 2. Switch to waiting state
                })
                .build();
    }

    private Receive waitingForReply() {
        return receiveBuilder()
                .match(ReplyMsg.class, reply -> {
                    System.out.println("Got reply: " + reply.getVal());
                    unstashAll();                            // 3. Restore postponed messages
                    getContext().become(idle());             // 4. Return to normal behavior
                })
                .matchAny(msg -> stash())                    // Postpone any other message while waiting
                .build();
    }

    public static Props props(ActorRef s) { return Props.create(AsyncClientActor.class, () -> new
AsyncClientActor(s)); }
}

// --- 4B. SYNCHRONOUS BLOCKING CLIENT (Patterns.ask + Await.result) ---
public class AskClientActor extends AbstractActor {
    private static final long TIMEOUT_MS = 5000;
    private final ActorRef server;
    public AskClientActor(ActorRef server) { this.server = server; }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
                .match(PutMsg.class, m -> server.tell(m, self()))
                .match(GetMsg.class, m -> {
                    // Blocks current thread until ReplyMsg arrives or 5s timeout expires
                    Future<Object> future = Patterns.ask(server, m, TIMEOUT_MS);
                    ReplyMsg reply = (ReplyMsg) Await.result(
                            future, scala.concurrent.duration.Duration.create(5, SECONDS));
                    System.out.println("Got sync reply: " + reply.getVal());
                })
                .build();
    }

    public static Props props(ActorRef s) { return Props.create(AskClientActor.class, () -> new
AskClientActor(s)); }
}
```

Pattern 4C — Non-Blocking Delayed Processing with `scheduler` + Busy Stash (Lab 2 `ex6`)

(Ref: Lab 2 `ex6/ServerActor.java` & `ex6/ReadyMessage.java`)

```
// Self-notification message carrying the original client ActorRef and the computed result
public class ReadyMessage implements Serializable {
    private final ActorRef sender;
    private final String result;
    public ReadyMessage(ActorRef sender, String result) {
        this.sender = sender;
        this.result = result;
    }
```

```
    public ActorRef getSender() { return sender; }
    public String getResult() { return result; }
}

// Server processes one TextMessage at a time with a 500ms non-blocking delay, stashing concurrent arrivals
public class AsyncProcessingServerActor extends AbstractActorWithStash {

    @Override
    public Receive createReceive() {
        // Ready/Idle state: accept the next TextMessage
        return receiveBuilder()
                .match(TextMessage.class, this::onTextMessage)
                .build();
    }

    private void onTextMessage(TextMessage message) {
        String upper = message.getText().toUpperCase();
        // 1. Switch to busy behavior so new requests arriving meanwhile are stashed
        getContext().become(onBusyMode());
        // 2. Schedule ReadyMessage to self() after 500ms WITHOUT blocking the thread
        //    Pass getSender() inside ReadyMessage because sender() when ReadyMessage arrives will be self()!
        getContext().getSystem().scheduler().scheduleOnce(
                Duration.ofMillis(500),
                self(),
                new ReadyMessage(getSender(), upper),
                getContext().dispatcher(),
                self()
        );
    }

    private Receive onBusyMode() {
        return receiveBuilder()
                .match(TextMessage.class, msg -> stash())     // Buffer new requests arriving while busy
                .match(ReadyMessage.class, this::onReady)     // Timer expired -> finish processing
                .build();
    }

    private void onReady(ReadyMessage msg) {
        unstashAll();                                         // 1. Prepend buffered requests back to mailbox
        getContext().become(createReceive());                 // 2. Become ready again
        msg.getSender().tell(new TextMessage(msg.getResult()), self()); // 3. Reply to original client
    }

    public static Props props() { return Props.create(AsyncProcessingServerActor.class); }
}
```

Pattern 5 — Fault Tolerance & Supervision Hierarchy (`resume` vs. `restart`)

(Ref: `SupervisionTemplate.java` & Lab 2 `ex5`)

```
// --- 5A. SUPERVISOR & SUPERVISED CHILD (Lecture 3 Slides 11-13 & Lab 2 Ex 5) ---
public class SupervisorActor extends AbstractActor {
    // Up to 10 faults within 1 minute (if exceeded -> child is stopped)
    private static final SupervisorStrategy strategy =
        new OneForOneStrategy( // Use AllForOneStrategy to apply directive to ALL children
            10,
            Duration.ofMinutes(1),
            DeciderBuilder
                .match(ArithmeticException.class, e -> SupervisorStrategy.resume())    // Keep state!
                .match(NullPointerException.class, e -> SupervisorStrategy.restart())  // Reset state!
                .match(IllegalArgumentException.class, e -> SupervisorStrategy.stop()) // Kill child!
                // Lab 2 Ex 5: use resume() to preserve state/avg, or restart() to reset state/map
                .match(Exception.class, e -> SupervisorStrategy.restart())
                .matchAny(o -> SupervisorStrategy.escalate())                          // Escalate to parent
                .build());

    @Override
    public SupervisorStrategy supervisorStrategy() { return strategy; }

    @Override
```

```
    public Receive createReceive() {
        // Spawns child inside supervisor's context and returns child ActorRef to sender
        return receiveBuilder()
                .match(Props.class, props -> sender().tell(getContext().actorOf(props), self()))
                .build();
    }
    public static Props props() { return Props.create(SupervisorActor.class); }
}

public class FaultyServerActor extends AbstractActor {
    private final Map<String, String> contacts = new HashMap<>(); // Preserved on resume(), wiped on
restart()!

    @Override
    public Receive createReceive() {
        return receiveBuilder()
                .match(PutMsg.class, this::onPut)
                .match(GetMsg.class, this::onGet)
                .build();
    }

    // Must declare 'throws Exception' if throwing checked exceptions!
    void onPut(PutMsg msg) throws Exception {
        if ("Fail!".equals(msg.getName())) {
            throw new Exception("Fault triggered by 'Fail!'"); // Handled by SupervisorActor!
        }
        contacts.put(msg.getName(), msg.getEmail());
    }

    void onGet(GetMsg msg) {
        sender().tell(new ReplyMsg(msg.getName(), contacts.get(msg.getName())), self());
    }

    @Override
    public void preRestart(Throwable reason, Optional<Object> msg) { System.out.print("Restarting ... "); }
    @Override
    public void postRestart(Throwable reason) { System.out.println("done! Map size = " + contacts.size()); }

    public static Props props() { return Props.create(FaultyServerActor.class); }
}

// --- MAIN: SPAWNING CHILD THROUGH SUPERVISOR ---
// scala.concurrent.duration.Duration timeout = scala.concurrent.duration.Duration.create(5, SECONDS);
// Future<Object> f = ask(supervisor, Props.create(FaultyServerActor.class), 5000);
// ActorRef server = (ActorRef) f.result(timeout, null);
```

Pattern 5B — Hierarchy of Responsibility (Safe Parent + Risky Child)

```
// Parent holds critical state that must NEVER be lost; child executes dangerous operations
public class SafeParentSupervisor extends AbstractActor {
    private final Map<String, Integer> criticalData = new HashMap<>(); // Safe across child crashes!
    private ActorRef worker;

    private static final SupervisorStrategy strategy = new OneForOneStrategy(
            10, Duration.ofMinutes(1),
            DeciderBuilder.match(Exception.class, e -> SupervisorStrategy.restart()).build());

    @Override
    public SupervisorStrategy supervisorStrategy() { return strategy; }

    @Override
    public void preStart() {
        worker = getContext().actorOf(RiskyWorker.props(), "worker");
    }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
                .match(TaskMsg.class, task -> worker.tell(task, self())) // Delegate risky task
                .match(ResultMsg.class, res -> criticalData.put(res.getId(), res.getValue()))
                .build();
```

```
    }
}
```

Pattern 6 — Remote Distribution over TCP

(Ref: `RemoteDistributionTemplate.java` & `remote.conf`)

```
# 1. File "remote.conf"
akka {
  actor { provider = remote }
  remote {
    enabled-transports = ["akka.remote.netty.tcp"]
    netty.tcp {
      hostname = "127.0.0.1"
      port = 6123
    }
  }
}
```

```
// 2. Server Main: start remote ActorSystem listening on 127.0.0.1:6123
Config conf = ConfigFactory.parseFile(new File("remote.conf"));
ActorSystem sys = ActorSystem.create("Server", conf);
sys.actorOf(ServerActor.props(), "serverActor");

// 3. Client Actor: lookup remote server by address using ActorSelection
public class RemoteClientActor extends AbstractActor {
    // Format: akka.tcp://<ActorSystemName>@<Hostname>:<Port>/user/<ActorName>
    private final ActorSelection server =
            getContext().actorSelection("akka.tcp://Server@127.0.0.1:6123/user/serverActor");

    @Override
    public Receive createReceive() {
        return receiveBuilder()
                .match(GetMsg.class, msg -> server.tell(msg, self())) // Identical to local tell()!
                .match(ReplyMsg.class, rep -> System.out.println("Remote reply: " + rep.getVal()))
                .build();
    }
}
```

Pattern 7 — Supervisor Dispatcher with Round-Robin & Load-Balancing (Evaluation Lab Pattern)

(Ref: `past-exam-2023` — Combines Supervision `resume()`, Dynamic Behavior `become()`, Round-Robin, and Sticky Load Balancing)

```
// 1. DISPATCHER ACTOR: Supervises processors AND switches routing policy via become()
public class DispatcherActor extends AbstractActor {
    private static final int NO_PROCESSORS = 2;
    private final List<ActorRef> processors = new ArrayList<>();
    // Sticky Load Balancer map: Sensor ActorRef -> assigned Processor ActorRef
    private final Map<ActorRef, ActorRef> sensorAssignment = new HashMap<>();
    private int rrIndex = 0;

    // Requirement: "When a processor receives a negative temperature, it fails.
    // The average before the failure must be retained, excluding the faulty reading." -> RESUME!
    private static final SupervisorStrategy strategy = new OneForOneStrategy(
            10, Duration.ofMinutes(1),
            DeciderBuilder.match(Exception.class, e -> SupervisorStrategy.resume()).build()
    );

    @Override
    public SupervisorStrategy supervisorStrategy() { return strategy; }

    @Override
    public void preStart() {
        // Spawn processors inside Dispatcher's context so Dispatcher supervises them!
        for (int i = 0; i < NO_PROCESSORS; i++) {
            processors.add(getContext().actorOf(SensorProcessorActor.props(), "p" + i));
```

```
        }
    }

    @Override
    public Receive createReceive() {
        return loadBalancer(); // Default initial policy
    }

    private Receive loadBalancer() {
        return receiveBuilder()
                .match(DispatchLogicMsg.class, m -> m.getLogic() == DispatchLogicMsg.ROUND_ROBIN,
                        m -> getContext().become(roundRobin()))
                .match(TemperatureMsg.class, this::dispatchDataLoadBalancer)
                .build();
    }

    private Receive roundRobin() {
        return receiveBuilder()
                .match(DispatchLogicMsg.class, m -> m.getLogic() == DispatchLogicMsg.LOAD_BALANCER,
                        m -> getContext().become(loadBalancer()))
                .match(TemperatureMsg.class, this::dispatchDataRoundRobin)
                .build();
    }

    // Sticky Load Balancing: permanently associate each sensor to the least-loaded processor
    private void dispatchDataLoadBalancer(TemperatureMsg msg) {
        ActorRef assigned = sensorAssignment.computeIfAbsent(msg.getSender(), s -> {
            ActorRef proc = processors.get(sensorAssignment.size() % processors.size());
            return proc;
        });
        assigned.tell(msg, self());
    }

    // Round Robin: forward each reading to the next processor in circular order
    private void dispatchDataRoundRobin(TemperatureMsg msg) {
        ActorRef proc = processors.get(rrIndex);
        rrIndex = (rrIndex + 1) % processors.size();
        proc.tell(msg, self());
    }

    public static Props props() { return Props.create(DispatcherActor.class); }
}

// 2. PROCESSOR ACTOR: Validates input BEFORE mutating state so resume() keeps previous average intact!
public class SensorProcessorActor extends AbstractActor {
    private double sum = 0;
    private int count = 0;
    private double currentAverage = 0;

    @Override
    public Receive createReceive() {
        return receiveBuilder().match(TemperatureMsg.class, this::gotData).build();
    }

    private void gotData(TemperatureMsg msg) throws Exception {
        // Check precondition FIRST: throw BEFORE updating sum/count so resume() excludes faulty value!
        if (msg.getTemperature() < 0) {
            throw new Exception("Negative temperature reading: " + msg.getTemperature());
        }
        sum += msg.getTemperature();
        count++;
        currentAverage = sum / count;
        System.out.println("Processor " + self().path().name() + " avg = " + currentAverage);
    }

    public static Props props() { return Props.create(SensorProcessorActor.class); }
}
```

5. Lab 1 & Lab 2 — Quick Exercise Summary Table

Lab & Exercise	Package	Core Pattern & Key Takeaway
Lab 1 — Base	`com.polimi.nsds.akka.base`	16 threads send 1000 increments → processed one by one atomically without locks.
Lab 1 — Ex 1A	`com.polimi.nsds.akka.ex1.classes`	Match by **distinct message classes** (`IncrementMsg` , `DecrementMsg`).
Lab 1 — Ex 1B	`com.polimi.nsds.akka.ex1.selectors`	Match by **predicate selector** (`.match(CounterMsg.class, m -> m.getOp() == ... , ... )`).
Lab 1 — Ex 2	`com.polimi.nsds.akka.ex2`	Non-negative counter (`AbstractActorWithStash`): `stash()` on `D` when `0` , `unstashAll()` on `I` .
Lab 1 — Ex 3	`com.polimi.nsds.akka.ex3`	Contact list request/reply: **Async** (`become(waiting)` + `stash()`) vs. **Sync** (`Patterns.ask` + `Await.result`).
Lab 2 — Ex 4	`com.polimi.nsds.akka.ex4`	**Sleep/Wakeup Echo Server**: `SleepMessage` → `become(onSleepMode())` + `stash()` ; `WakeupMessage` → `unstashAll()` + `become(createReceive())` .
Lab 2 — Ex 5	`com.polimi.nsds.akka.ex5`	**Supervision (`resume` vs. `restart`)**: `PutMsg("Fail!")` throws `Exception` ; `SupervisorActor` (`OneForOneStrategy`) verifies `resume()` keeps contacts while `restart()` resets the `HashMap` (preserving `ActorRef` & mailbox).
Lab 2 — Ex 6	`com.polimi.nsds.akka.ex6`	**Non-Blocking Delayed Server**: `become(onBusyMode())` + `scheduler().scheduleOnce(500ms, self(), new ReadyMessage(getSender(), upper), ... )` + `stash()` for parallel requests.